

Requirement Analysis

Customer Journey Map

Date: 01 October 2026 | Team ID: SWTID-2026-4057 | Project Name: Implement Client Script & UI Policy (Incident)

Customer / User: ServiceNow Incident user who creates, updates, validates, and manages Incident records.

Journey Objective: Understand how an Incident user moves from entering Incident information to completing a valid update while UI Policies and Client Scripts guide field behavior and prevent incorrect actions.

Journey Stage	User Goal / Action	Touchpoint & System Response	Thoughts / Feelings	Pain Points	Opportunity / Project Response
1. Open / Create Incident	Create a new Incident or open an existing Incident for updating.	Incident form loads. UI Policy rules are available to control fields according to Incident conditions.	"I want the form to clearly guide me." Feels confident when the form behaves predictably.	Without controls, users may not know which fields require special attention.	Use UI Policy on the Incident table to apply conditional field behavior at the form level.
2. Enter Incident Details	Enter and update Incident information, including Impact and other relevant fields.	The user interacts directly with Incident form fields. Client Scripts can respond immediately to field changes.	"I should not have to remember every rule manually."	Manual entry can lead to incomplete or inconsistent information.	Provide dynamic field behavior and automatic updates using UI Policies and Client Scripts.
3. Set Impact to High	Set or change Impact to 1 – High when the Incident has high impact.	High Impact Control UI Policy activates. Assignment group becomes mandatory; Urgency is controlled. The onChange script sets Urgency to High.	"The system is applying the correct rules automatically." Feels guided rather than interrupted.	Important fields may otherwise remain uncontrolled under high-impact conditions.	Apply conditional mandatory/read-only behavior and automatically set Urgency to High.
4. Attempt to Save	Save the Incident after entering the required information.	The onSubmit Client Script checks the save condition. A High Impact Incident cannot be saved when Assigned To is empty.	"I need to know what is missing before I can continue."	Invalid or incomplete submissions can affect Incident handling and routing.	Prevent invalid submission and display clear validation feedback to the user.
5. Edit Through Incident List	Update an Incident from the list when necessary.	The onCellEdit Client Script prevents direct State changes through Incident list editing and displays a guidance message.	"I should know why this action is blocked and where to make the change."	Uncontrolled list editing may bypass the intended Incident form interaction.	Restrict unsupported State changes and guide the user to the appropriate Incident form.
6. Complete Valid Update	Finish the Incident update with the required information and permitted actions.	UI Policies and Client Scripts have guided the user, controlled fields, validated the submission, and restricted unsupported interactions.	"The form helps me complete the Incident correctly." Feels more confident about data quality.	Without these controls, users may depend on memory and manual checking.	Deliver a guided Incident experience that improves consistency, reduces incorrect data entry, and supports reliable Incident processing.

Customer Journey Insight	Key Project Response
Need: A clear, guided Incident form that responds to user actions and conditions.	UI Policies provide conditional field control, while Client Scripts provide immediate automation, validation, and interaction control.
Expected Experience: Users should understand what is required, what is automatically controlled, and why an action may be blocked.	The implementation makes Assignment group mandatory for High Impact, controls Urgency, validates High Impact submissions, and restricts direct State changes through list editing.

Conclusion: The journey shows that the Incident user benefits from controls being applied at the point of interaction. The combined use of UI Policies, UI Policy Actions, and Client Scripts reduces reliance on manual checking and supports more consistent Incident data entry and updates.